

AEM Code Quality

Study Guide for Beginners

The rules, patterns, and pitfalls Cloud Manager checks before your code reaches production.

Volume 3 of the AEM study material series.

Table of contents

Table of contents	2
1. Introduction	5
The eight categories at a glance	5
2. Why code quality matters in AEM.....	6
Real-world failure modes	6
3. SonarQube — Critical bugs.....	7
java:S2095 — Resources should be closed	7
java:S2885 — Thread safety in OSGi services	7
java:S2259 — Null pointer dereference	7
java:S2068 — Hardcoded credentials	7
4. SonarQube — Code smells.....	9
Complexity and structure	9
Exception handling	9
Logging and APIs	9
Magic values and duplication.....	9
5. AEM best practices	11
AEM-BP-001 — ResourceResolver lifecycle	11
AEM-BP-002 — Service user pattern	11
AEM-BP-004 — Sling Models for business logic	11
AEM-BP-006 — High-level APIs.....	11
AEM-BP-008 — Externalize paths.....	11
OSGi lifecycle annotations.....	11
6. OakPAL — Content package rules.....	13
OAKPAL-001 — Scoped filter roots (Critical).....	13
OAKPAL-002 — No writes to /libs (Blocker)	13
oakpal:RequiredPrimaryType (Error)	13
oakpal:ImmutableApps (Warning).....	13
OAKPAL-005 — Overlapping filters (Major).....	13
OAKPAL-006 — rep:policy exclusion (Critical)	13
OAKPAL-007 — No /var in packages (Major).....	13
OAKPAL-008 — Clientlib category naming (Minor).....	13
7. HTL / Sightly rules.....	15
HTL-001 — Never use context='unsafe'	15
HTL-002 — data-sly-use for business logic	15
HTL-003 — context='uri' for href values.....	15

HTL-004 — JavaScript context escaping	15
Key HTL attributes	15
HTL-005 — Client libraries	15
8. OSGi / Felix rules	17
OSGI-001 — Explicit Import-Package declarations (Critical)	17
OSGI-002 — Maven scope 'provided' for container APIs (Blocker)	17
@Reference injection	17
Reference cardinality	17
OSGI-005 — OSGi version ranges	17
OSGI-006 — Context classloader (Critical)	17
Sling Scheduler for background tasks	17
@Deactivate lifecycle	17
9. Dispatcher rules	19
DISP-001 — Request validation (Critical)	19
DISP-003 — Author isolation (Critical)	19
DISP-004 — Block dangerous default selectors (Critical)	19
Dispatcher filter — deny by default	19
/allowedClients	19
DISP-005 — Security headers (Major)	19
Authentication and caching	19
DISP-006 — Query string caching	19
10. Performance optimization	21
PERF-001 & PERF-002 — Unbounded JCR queries (Critical)	21
PERF-005 — Index type selection	21
PERF-003 — session.save() in loops (Major)	21
PERF-006 — Lazy loading	21
Dispatcher caching	21
11. Security rules	23
SEC-001 — JCR-SQL2 injection (Critical)	23
SEC-002 — Default password (Blocker)	23
SEC-003 — CSRF protection (Critical)	23
SEC-004 — Sling Servlet binding (Critical)	23
SEC-005 — TLS/HTTPS (Blocker)	23
SEC-006 — User audit (Critical)	23
12. Severity levels and Cloud Manager	25
Severity scale	25

Key Blocker rules to memorize25

Key Critical rules25

Cloud Manager integration25

13. Closing.....27

 How to remember them27

 For the exam, focus on.....27

1. Introduction

This guide recaps every code quality rule that applies to AEM development.

It covers what Cloud Manager enforces automatically before code reaches production, plus the standards every AEM developer is expected to follow in their day-to-day work.

The material is organized into eight categories with hundreds of rules and concepts. The goal of this guide is to make the most important patterns clear and easy to recall.

The eight categories at a glance

- SonarQube — Java code correctness and maintainability.
- AEM Best Practices — AEM-specific Java patterns (resources, service users, Sling Models, high-level APIs).
- OakPAL — content package structure and repository correctness.
- HTL / Sightly — template security and XSS prevention.
- OSGi / Felix — bundle modularity and service lifecycle.
- Dispatcher — HTTP boundary, security headers, and cache control.
- Performance Optimization — JCR queries, Oak indexes, and efficient data access.
- Security — injection, authentication, CSRF, TLS, and least privilege.

The eight categories of AEM code quality

A roadmap of what this guide covers

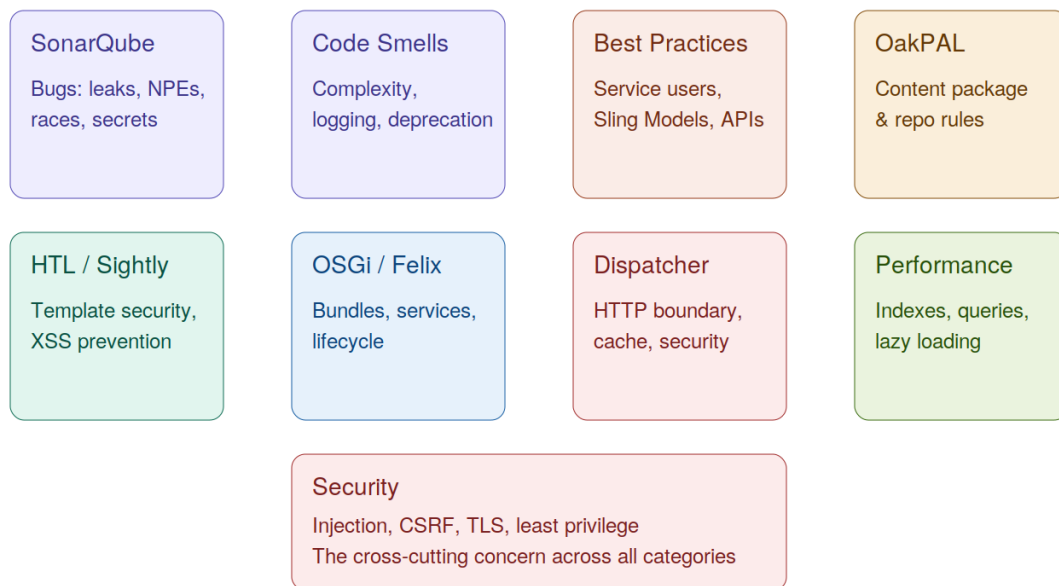


Figure 1. The eight categories of AEM code quality.

2. Why code quality matters in AEM

AEM applications run in production environments with thousands of concurrent users. Bad code patterns in AEM cause specific, serious problems.

Real-world failure modes

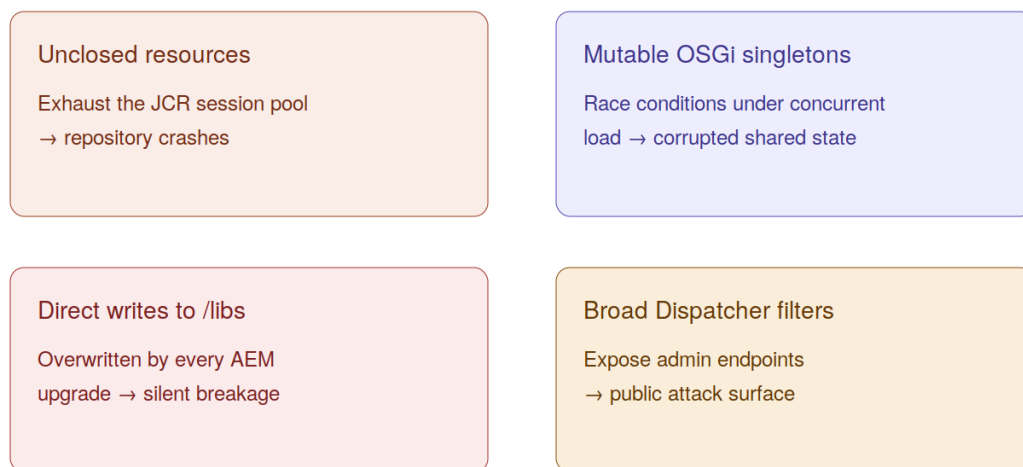
- Unclosed resources exhaust the JCR session pool and crash the repository.
- OSGi singletons with mutable state create race conditions under concurrent load.
- Direct writes to /libs break on every AEM upgrade.
- Overly broad Dispatcher filters expose admin endpoints to the internet.

Code quality in AEM is not just about clean code — it is about system stability and security.

Cloud Manager enforces a subset of these rules automatically through SonarQube, OakPAL, and the AEM Analyzer Plugin. The rest are enforced through code review and developer discipline.

Why code quality matters in AEM

Four bad patterns that cause real production failures



Code quality in AEM is not just style — it protects system stability and security.

Cloud Manager enforces a subset automatically; the rest depends on developer discipline.

Figure 2. Four bad patterns that cause real production failures.

3. SonarQube — Critical bugs

These are the rules that cause production incidents if violated. Memorize them by their rule key.

java:S2095 — Resources should be closed

- ResourceResolver, JCR Session, and InputStream all implement AutoCloseable.
- An unclosed ResourceResolver leaks a JCR session, eventually exhausting the connection pool.
- Fix: always use try-with-resources.

java:S2885 — Thread safety in OSGi services

- OSGi @Component services are singletons shared across all concurrent request threads.
- Mutable instance fields (like ArrayList or HashMap) create race conditions.
- Fix: use ConcurrentHashMap, CopyOnWriteArrayList, or keep the service stateless.

java:S2259 — Null pointer dereference

- AEM APIs like adaptTo(), getChild(), and getResource() frequently return null.
- Calling any method on a null reference throws a NullPointerException at runtime.
- Fix: always null-check results from Sling and JCR APIs before use.

java:S2068 — Hardcoded credentials

- Passwords and API keys in source code are visible to anyone with repository access.
- Fix: use OSGi configuration with AEM Crypto Support, or Cloud Manager environment variables.

SonarQube — Critical bug rules

Memorize these by rule key — they cause production incidents

java:S2095 Resources should be closed

Unclosed ResourceResolver leaks JCR sessions

Fix: try-with-resources

java:S2885 Thread safety in OSGi services

Mutable fields in singletons cause race conditions

Fix: ConcurrentHashMap or stateless service

java:S2259 Null pointer dereference

adaptTo(), getChild(), getResource() may return null

Fix: always null-check Sling/JCR results

java:S2068 Hardcoded credentials

Passwords/keys in source code are visible to anyone

Fix: OSGi config + Crypto Support, or env vars

Figure 3. The four SonarQube critical bug rules to memorize.

4. SonarQube — Code smells

These rules address maintainability, clarity, and reliability in Java code.

Complexity and structure

- java:S1541 — Methods too complex. Cyclomatic complexity counts independent execution paths (if, for, while, case, catch). Default threshold is 10. Above this, the method is too hard to test and maintain. Fix: extract private helper methods, use early returns.
- java:S2972 — Classes too long. A class mixing rendering, business logic, and data access violates Single Responsibility. Fix: a Sling Model handles view concerns; OSGi services handle business logic.

Exception handling

- java:S00108 — Empty catch blocks. An empty catch silently swallows exceptions: no log, no rethrow, no recovery. This hides failures in JCR saves, workflow completions, and replication events. Minimum fix: `log.error("message", e)`.
- java:S1166 — Exceptions should not be ignored. Similar to S00108 but also catches cases where the exception variable is declared but never used. The exception must be logged, rethrown, or explicitly handled.

Logging and APIs

- java:S106 — Use SLF4J instead of `System.out`. `System.out` bypasses the Sling Log Service, so output is not routed, persisted, or throttled correctly. Fix: use `Logger log = LoggerFactory.getLogger(getClass())` and parameterized messages.
- java:S1874 — Deprecated API usage. `getAdministrativeResourceResolver()` was removed in AEMaaCS — use `getServiceResourceResolver()` with a service user. `WCMUsePojo` is legacy — replace with Sling Models.

Magic values and duplication

- java:S1192 — String literal duplication. JCR property names like `"jcr:title"` repeated in multiple places should be extracted to constants. A typo in one occurrence causes silent data loss.
- java:S109 — Magic numbers. Inline numbers like 100 in `"results.size() > 100"` have no self-documenting name. Fix: `private static final int MAX_SEARCH_RESULTS = 100`.

SonarQube — Code smell rules

Maintainability, clarity, and reliability rules

java:S1541 Methods too complex Cyclomatic > 10 — extract helpers	java:S00108 Empty catch blocks Hides JCR/workflow failures — log it
java:S1166 Exceptions ignored Variable declared, never used — handle it	java:S106 System.out logging Bypasses Sling Log Service — use SLF4J
java:S1874 Deprecated APIs WCMUsePojo, getAdministrativeRR()	java:S1192 String literal duplication 'jcr:title' repeated → extract constant
java:S109 Magic numbers Inline 100, 500 → named constants	java:S2972 Classes too long Mixed concerns → split responsibilities

Figure 4. The eight SonarQube code smell rules in AEM development.

5. AEM best practices

These are AEM-specific patterns that go beyond standard Java quality.

AEM-BP-001 — ResourceResolver lifecycle

- Every ResourceResolver obtained from ResourceResolverFactory must be closed in the same method.
- Use try-with-resources — it guarantees closure even on exceptions.

AEM-BP-002 — Service user pattern

- getAdministrativeResourceResolver() grants unrestricted super-admin access with no audit trail.
- Replaced by getServiceResourceResolver() with a system user that has only the permissions it needs.
- The mapping is declared in the ServiceUserMapperImpl.amended OSGi factory configuration.

AEM-BP-004 — Sling Models for business logic

- HTL is intentionally limited — it cannot execute arbitrary Java.
- Business logic belongs in @Model classes, not in HTL scripts or Servlets.
- Sling Models are testable with AEM Mocks without a running AEM instance.

AEM-BP-006 — High-level APIs

- Use PageManager, TagManager, and AssetManager instead of raw JCR node creation.
- These APIs apply AEM's internal business rules, node structures, and event triggers.
- Raw session.addNode() bypasses all of this.

AEM-BP-008 — Externalize paths

- Never hardcode paths like "/content/mysite/en" in Java.
- Use OSGi configuration properties or Context-Aware Configurations (CAConfig).
- CAConfig stores settings per site in /conf and resolves them automatically from the content context.

OSGi lifecycle annotations

- @Activate — called when all @Reference dependencies are bound.
- @Modified — called when configuration changes, without restarting the component.
- @Deactivate — release all resources: close resolvers, stop threads, remove listeners.

AEM best practices

AEM-specific patterns beyond standard Java rules

AEM-BP-001 ResourceResolver lifecycle
Every resolver from ResourceResolverFactory must be closed in the same method — use try-with-resources

AEM-BP-002 Service user pattern
Replace getAdministrativeResourceResolver() with getServiceResourceResolver() + ServiceUserMapper config

AEM-BP-004 Sling Models for business logic
HTL is intentionally limited — logic belongs in @Model classes, testable with AEM Mocks

AEM-BP-006 High-level APIs
PageManager, TagManager, AssetManager apply AEM business rules — raw session.addNode() bypasses them

AEM-BP-008 Externalize paths
Never hardcode /content/mysite/en — use OSGi properties or Context-Aware Configurations (CAConfig in /conf)

OSGi lifecycle: @Activate on bind • @Modified on config change • @Deactivate to release resources

Figure 5. The core AEM best practice patterns for Java code.

6. OakPAL — Content package rules

OakPAL validates content packages for structural and architectural correctness.

OAKPAL-001 — Scoped filter roots (Critical)

- A filter of `/apps/myproject` deletes everything under that path on install.
- Fix: use specific child paths — `/apps/myproject/components`, `/apps/myproject/templates`.

OAKPAL-002 — No writes to `/libs` (Blocker)

- `/libs` is the immutable AEM platform layer — any change is overwritten by the next AEM update.
- Fix: use the overlay pattern — copy the resource to the same relative path under `/apps`.
- The Sling Resource Merger resolves `/apps` before `/libs`, so the overlay takes precedence.

`oakpal:RequiredPrimaryType` (Error)

- Every JCR node must explicitly declare its `jcr:primaryType`.
- Common types: `cq:Page`, `cq:Component`, `nt:unstructured`, `sling:Folder`, `cq:ClientLibraryFolder`.

`oakpal:ImmutableApps` (Warning)

- `/apps` is for code and structural definitions, not for runtime content values.
- Page content and authored data belong in `/content`.

OAKPAL-005 — Overlapping filters (Major)

- If one filter root is an ancestor of another, install order is undefined — one may delete what the other deployed.
- Fix: use non-overlapping sibling paths.

OAKPAL-006 — `rep:policy` exclusion (Critical)

- `rep:policy` nodes define JCR ACLs.
- Accidentally packaging them overwrites production security settings on every deployment.

OAKPAL-007 — No `/var` in packages (Major)

- `/var` holds runtime data: event queues, audit logs, workflow payloads, Oak indexes.
- Packaging `/var` freezes dynamic data into a static snapshot — do not include it.

OAKPAL-008 — Clientlib category naming (Minor)

- Generic names like `"utils"` or `"common"` conflict with AEM Core Components and third-party libraries.
- Fix: namespace with the project name — `"myproject.components.header"`.

OakPAL — Content package rules

Validates packages for repository correctness

OAKPAL-001	Scoped filter roots	<i>Critical</i>
/apps/myproject deletes everything — use child paths		
OAKPAL-002	No writes to /libs	<i>Blocker</i>
/libs is immutable — overlay to /apps instead		
RequiredPrimaryType	Every node needs jcr:primaryType	<i>Error</i>
cq:Page, cq:Component, nt:unstructured, sling:Folder		
ImmutableApps	/apps for code, not content	<i>Warning</i>
Authored data and runtime values belong in /content		
OAKPAL-005	No overlapping filters	<i>Major</i>
Ancestor + descendant = undefined install order		
OAKPAL-006	Exclude rep:policy nodes	<i>Critical</i>
Packaging ACLs overwrites production security		
OAKPAL-007	No /var in packages	<i>Major</i>
Runtime data — queues, audits, indexes — never freeze it		

Figure 6. OakPAL rules tagged by severity level.

7. HTL / Sightly rules

HTL's security model relies on automatic output escaping. Breaking it introduces XSS vulnerabilities.

HTL-001 — Never use context='unsafe'

- context='unsafe' disables all escaping — raw HTML is written directly to the output.
- An attacker can inject `<script>alert(1)</script>` and it executes in the browser.
- context='html' is the correct alternative for RTE-sourced rich text — it applies Anti-Samy sanitization.

HTL-002 — data-sly-use for business logic

- HTL enforces strict separation between presentation and logic.
- Business logic goes in Java Sling Models accessed via data-sly-use.model="com.myproject.MyModel".

HTL-003 — context='uri' for href values

- Without context='uri', an attacker can inject `javascript:alert(1)` as an href.
- context='uri' validates URI schemes and strips dangerous protocols like javascript: and data:.

HTL-004 — JavaScript context escaping

- For values inside JavaScript event handlers, use context='scriptToken' or context='scriptString'.
- Default HTML attribute escaping does not protect against JavaScript injection.

Key HTL attributes

- data-sly-test — conditional rendering; falsy values remove the element and its entire subtree.
- data-sly-list — iteration; provides itemList.index, itemList.first, itemList.last, itemList.odd.
- data-sly-unwrap — renders children while removing the wrapper element tag from the output.
- data-sly-include — includes another HTL script in the current context.
- data-sly-resource — dispatches a new Sling rendering for a JCR resource by path.

HTL-005 — Client libraries

- Inline CSS or JavaScript in HTL is not cached and cannot be combined with other assets.
- Use cq:ClientLibraryFolder, referenced via data-sly-call="{clientlib.all @categories='...'}".

HTL / Sightly rules

Output escaping is the core security model

Security contexts (always escape)

context='unsafe'	Disables all escaping — XSS hole. Never use it.
context='html'	Sanitizes RTE rich text via Anti-Samy. The right choice.
context='uri'	Validates URI scheme. Blocks javascript: and data:
context='scriptToken'	Use inside JavaScript event handlers and scripts.

Key HTL attributes

data-sly-test	Conditional rendering (falsy removes subtree)
data-sly-list	Iteration with itemList.index/first/last/odd
data-sly-use	Bind a Sling Model for business logic
data-sly-resource	Dispatch a new Sling rendering by path

Figure 7. HTL security contexts and key data-sly attributes.

8. OSGi / Felix rules

OSGi provides strict module isolation. Violating its rules causes runtime failures that are hard to diagnose.

OSGI-001 — Explicit Import-Package declarations (Critical)

- Wildcard imports resolve lazily — the bundle activates but crashes with `ClassNotFoundException` at runtime.
- Explicit imports fail fast at install time if a dependency is missing.

OSGI-002 — Maven scope 'provided' for container APIs (Blocker)

- `servlet-api`, `jsp-api`, and `osgi.core` must be `scope="provided"`.
- `Scope="compile"` bundles them inside the JAR, causing `ClassCastException` from classloader conflicts.

@Reference injection

- Use `@Reference` annotation to inject OSGi services — DS manages the full bind/unbind lifecycle.
- Avoid using `BundleContext.getService()` directly — it bypasses lifecycle management.

Reference cardinality

- **MANDATORY** (default) — the component won't activate if the service is absent.
- **OPTIONAL** — the component activates; the injected field is null and must be null-checked.

OSGI-005 — OSGi version ranges

- Explicit version ranges prevent accidentally binding to a major API upgrade.
- Format: `[3.0,4.0)` means greater or equal to 3.0 and less than 4.0.

OSGI-006 — Context classloader (Critical)

- `Thread.currentThread().getContextClassLoader()` returns the caller's classloader.
- In OSGi, each bundle has its own classloader — the caller's loader cannot see this bundle's classes.
- Fix: use `SomeClass.class.getClassLoader()` instead.

Sling Scheduler for background tasks

- Never use raw `Thread` or `ScheduledExecutorService` — they survive bundle restarts.
- Implement `Runnable` and register with the Sling Scheduler via `@Component(service=Runnable.class)`.

@Deactivate lifecycle

- Release all resources when the component shuts down: close `ResourceResolvers`, stop background threads, remove event listeners.

OSGi / Felix rules

Module isolation, services, classloaders, lifecycle

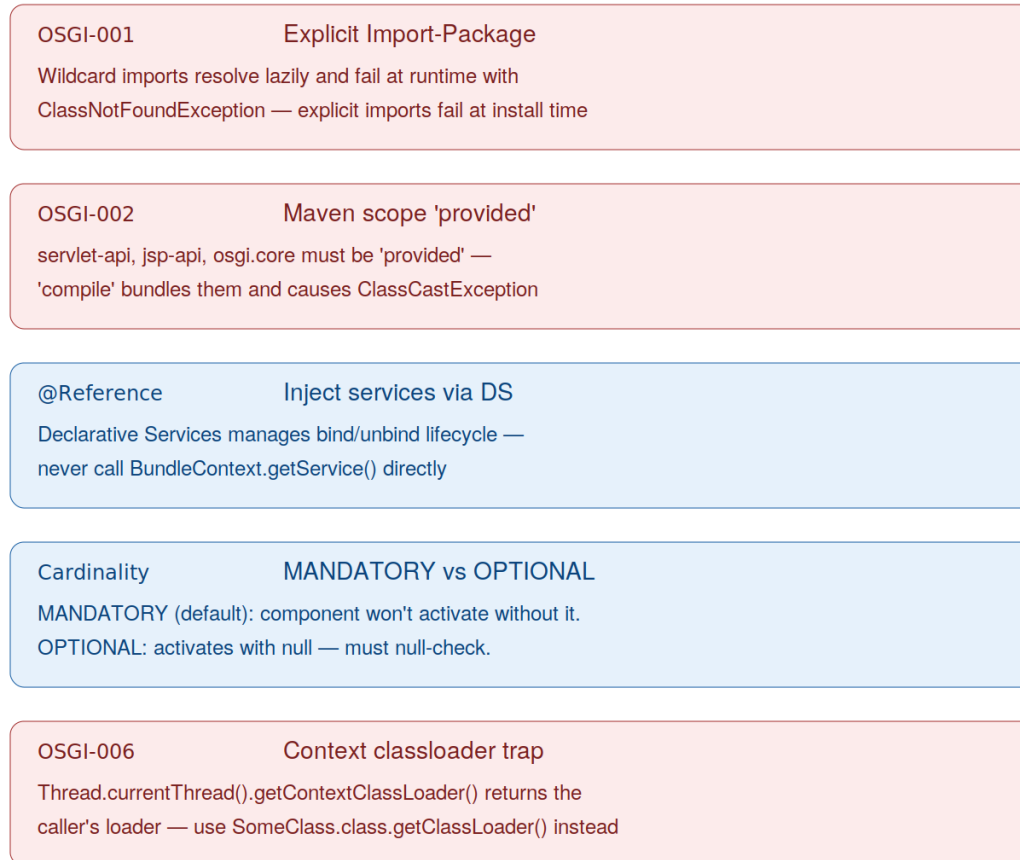


Figure 8. Core OSGi rules: imports, scopes, references, classloaders.

9. Dispatcher rules

The Dispatcher is AEM's first security boundary. Its configuration must be explicitly restrictive.

DISP-001 — Request validation (Critical)

- Block path traversal sequences (`/../`, `/./`) to prevent directory traversal attacks.
- Allowlist only required HTTP methods (GET, POST) — reject DELETE, PUT, TRACE.

DISP-003 — Author isolation (Critical)

- The AEM Author service (port 4502) must never be accessible through the Dispatcher.
- Author exposes CRXDE Lite, Package Manager, Felix Web Console, and all administrative endpoints.
- Only Publisher instances (port 4503) serve public traffic.

DISP-004 — Block dangerous default selectors (Critical)

- `/*.infinity.json` returns the full JCR subtree as JSON — block it.
- `/*.json` should be denied unless specifically required for API endpoints.
- `/bin/*` endpoints including QueryBuilder (`/bin/querybuilder.json`) must be denied.

Dispatcher filter — deny by default

- Start with a rule that denies everything.
- Then explicitly allow only the paths that need to be accessible.

`/allowedClients`

- Only AEM Publisher IPs should be allowed to send cache invalidation requests to `/dispatcher/invalidate.cache`.

DISP-005 — Security headers (Major)

- Inject X-Frame-Options: SAMEORIGIN to prevent clickjacking.
- Also apply Content-Security-Policy, X-Content-Type-Options, and Strict-Transport-Security.

Authentication and caching

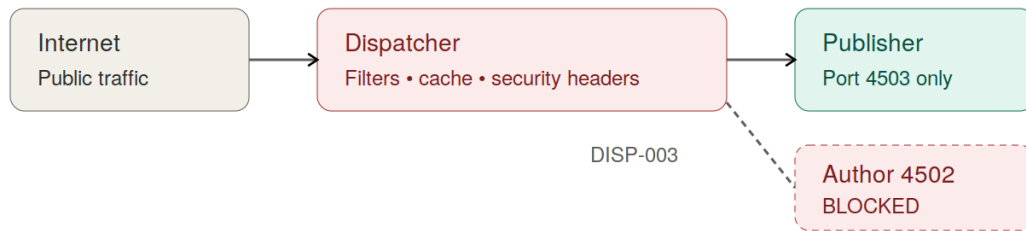
- The Dispatcher cannot differentiate between users — never cache authenticated responses.
- Configure `/auth_checker` or exclude authenticated paths from cache rules.

DISP-006 — Query string caching

- Unrestricted query string caching enables cache busting DoS attacks.
- Use `/ignoreUrlParams` to allowlist known parameters; all others are stripped for caching.

Dispatcher rules

AEM's first security boundary — deny by default



Critical Dispatcher rules

DISP-001	Request validation
Block ../../ path traversal and reject DELETE/PUT/TRACE	
DISP-004	Block default selectors
Deny /*.infinity.json, /*.json, /bin/*, /bin/querybuilder.json	
DISP-005	Security headers
Inject X-Frame-Options, CSP, X-Content-Type-Options, HSTS	
DISP-006	Query string allowlist
Use /ignoreUrlParams — unrestricted = cache-busting DoS	

Filter principle: deny everything, then allowlist only required paths.

Figure 9. Dispatcher topology: Internet → Dispatcher → Publisher (Author blocked).

10. Performance optimization

AEM performance issues fall into a few predictable patterns. These are the most common.

PERF-001 & PERF-002 — Unbounded JCR queries (Critical)

- A JCR query without a matching Oak index causes a full repository traversal.
- Full traversals scale $O(n)$ with repository size — they are catastrophic under load.
- Diagnose with the AEM Query Performance tool and Oak QueryExplain.
- Fix: create an Oak index definition (oak:QueryIndexDefinition) and deploy it as code.

PERF-005 — Index type selection

- Property indexes (type='property') are efficient for exact-match equality queries.
- Lucene indexes are for full-text search only — they are larger and slower for equality lookups.
- Scope Lucene indexes with 'includedPaths' to avoid indexing the entire repository.

PERF-003 — session.save() in loops (Major)

- Each session.save() creates a transaction boundary with disk I/O and Lucene index updates.
- Calling it once per item in a 100,000-node bulk import is catastrophically slow.
- Fix: batch changes and save every 500–1000 items.

PERF-006 — Lazy loading

- resource.listChildren() returns a lazy Iterator backed by the JCR NodeIterator.
- Converting it to a new ArrayList immediately loads all children into memory.
- For large content trees, iterate lazily and never collect unless the full list is required.

Dispatcher caching

- A cached HTML response is served in less than 5 ms directly from disk.
- An uncached AEM request takes 50–500 ms through the full rendering pipeline.
- Cache aggressively, invalidate precisely.

Performance optimization

Common AEM bottlenecks and the speed difference

Cached vs uncached request

Cached HTML (Dispatcher) Served directly from disk < 5 ms	Uncached AEM render Full Sling pipeline 50 - 500 ms
--	--

Critical performance rules

PERF-001/002	Unbounded JCR queries No matching Oak index = full traversal, $O(n)$ of repo size Diagnose with QueryExplain + fix with oak:QueryIndexDefinition
PERF-005	Index type selection Property index for equality lookups; Lucene only for full-text Always scope with includedPaths — never index whole repo
PERF-003	session.save() in loops Each save = transaction + disk + Lucene update Batch every 500–1000 items in bulk imports
PERF-006	Lazy loading listChildren() returns a lazy iterator — don't collect into ArrayList unless the full list is truly required

Figure 10. The 100x speed difference between cached and uncached requests.

11. Security rules

These are the patterns that protect AEM from attack.

SEC-001 — JCR-SQL2 injection (Critical)

- Never build queries via string concatenation with user-supplied values.
- An attacker can inject WHERE clause conditions to retrieve arbitrary repository content.
- Fix: use parameterized queries via QueryManager with variables set separately from the query.

SEC-002 — Default password (Blocker)

- The default "admin" password is checked by automated scanners within seconds.
- AEMaaCS provisions randomized admin passwords automatically.
- Cloud Manager Health Checks block environment promotion if the default password is still in use.

SEC-003 — CSRF protection (Critical)

- Cross-Site Request Forgery forces logged-in users to submit state-changing requests unknowingly.
- AEM's Granite CSRF Filter validates a token on all POST/DELETE/PUT requests.
- Custom API endpoints must include the :cq_csrf_token parameter in the request.

SEC-004 — Sling Servlet binding (Critical)

- Servlets registered without explicit resource type binding can be triggered on any content node.
- Always bind servlets to a specific resource type:
resourceTypes="myproject/components/product".

SEC-005 — TLS/HTTPS (Blocker)

- TLS provides confidentiality, integrity, and server authentication.
- All AEM environments — Author, Publish, Dispatcher — must use HTTPS.

SEC-006 — User audit (Critical)

- Review all active accounts under /home/users regularly.
- Verify service users have minimal permissions (principle of least privilege).
- Disable or remove inactive author accounts.

Security rules

Patterns that protect AEM from real attacks

SEC-001 Critical JCR-SQL2 injection Never concatenate user input into queries. Use parameterized queries via QueryManager.	SEC-002 Blocker Default admin password Auto-scanned within seconds. AEMaaCS randomizes it; Cloud Manager blocks promotion.
SEC-003 Critical CSRF protection Granite CSRF Filter validates tokens. POST/DELETE/PUT must include :cq_csrf_token.	SEC-004 Critical Servlet binding Bind to specific resourceTypes — never let a Servlet match any content node.
SEC-005 Blocker TLS / HTTPS everywhere Author, Publish, Dispatcher all HTTPS. Confidentiality + integrity + auth.	SEC-006 Critical User audit Review /home/users regularly. Service users get least privilege. Disable inactive.

Defense in depth: each layer assumes the others may fail.

Figure 11. The six security rules and their severity in Cloud Manager.

12. Severity levels and Cloud Manager

Severity levels determine how Cloud Manager responds to violations.

Severity scale

- Blocker — build stops immediately, deployment is blocked, cannot be overridden.
- Critical — hard stop in the pipeline, code must be fixed.
- Important — pipeline pauses; a Deployment Manager or Business Owner can override and continue.
- Major — flagged in reporting, tracked but does not block by default.
- Minor / Warning — informational.

Key Blocker rules to memorize

- OAKPAL-002: writing to /libs.
- SEC-002: default admin password.
- SEC-005: no TLS.
- OSGI-002: container APIs bundled with compile scope.

Key Critical rules

- S2095 (resources not closed), S2885 (thread safety), S2259 (null pointer), S2068 (hardcoded credentials).
- OAKPAL-001 (broad filter roots), OAKPAL-006 (rep:policy packaging).
- DISP-001 (no request validation), DISP-003 (Author exposed), DISP-004 (JSON endpoints open).
- SEC-001 (query injection), SEC-003 (no CSRF), SEC-004 (servlet binding).

Cloud Manager integration

- SonarQube runs in the Code Quality pipeline step.
- The AEM Analyzer Maven Plugin runs in the Build step and catches AEMaaS compatibility issues.
- SonarQube behavior can be customized with a sonar-project.properties file at the repository root.
- JaCoCo measures code coverage; the default Important failure threshold is 50% line coverage.

Severity levels & Cloud Manager

How violations affect the deployment pipeline

Blocker	Build stops. Cannot be overridden. <i>e.g. OAKPAL-002, SEC-002, SEC-005, OSGI-002</i>
Critical	Hard stop — code must be fixed. <i>e.g. S2095, S2885, OAKPAL-001, DISP-003, SEC-001</i>
Important	Pipeline pauses; can be overridden. <i>e.g. By Deployment Manager or Business Owner</i>
Major	Flagged in reporting; does not block. <i>e.g. OAKPAL-005, OAKPAL-007, DISP-006, PERF-003</i>
Minor / Warning	Informational only. <i>e.g. OAKPAL-008 (clientlib naming) and similar</i>

Cloud Manager runs SonarQube + AEM Analyzer + JaCoCo (50% line coverage default).
Customize via sonar-project.properties at the repo root.

Figure 12. The five severity levels and how Cloud Manager handles each.

13. Closing

Code Quality in AEM spans eight categories, each protecting a different dimension of the system.

How to remember them

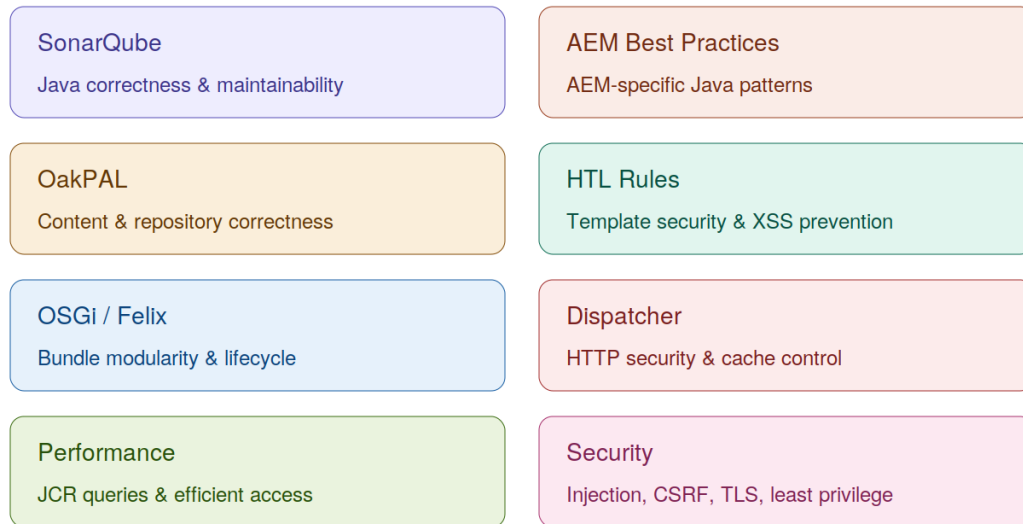
- SonarQube Rules — Java code correctness and maintainability.
- AEM Best Practices — AEM-specific Java patterns (resources, service users, Sling Models, APIs).
- OakPAL — content package structure and repository correctness.
- HTL Rules — template security and XSS prevention.
- OSGi / Felix — bundle modularity and lifecycle correctness.
- Dispatcher Rules — HTTP security and cache control.
- Performance Optimization — JCR queries, Oak indexes, and efficient data access.
- Security — injection, authentication, CSRF, TLS, and least privilege.

For the exam, focus on

- The Critical and Blocker rules by category.
- The fix pattern for each rule, not just the problem.
- The difference between Critical (hard stop) and Important (overridable) in Cloud Manager gates.
- The overlay pattern for /libs and the service user pattern for resource access.

Closing — what to remember

Eight categories, one purpose: a stable, secure AEM



For the exam, focus on:

- The Critical and Blocker rules by category, with their fix patterns
- Critical (hard stop) vs Important (overridable) in Cloud Manager gates
- The overlay pattern for /libs and the service user pattern for resources

Figure 13. Recap of the eight categories with the key exam focus areas.